

Robot Structural Proprioception

Pass 1 — Conceptual Foundation

Dandelion Engineering

July 16, 2026

Contents

1	Introduction	2
2	Domain Background: the problem, and why it is hard	2
2.1	The problem: a body that changes under a controller that assumes it doesn't	2
2.2	Why it is hard, part 1: some changes are invisible in principle	3
2.3	Why it is hard, part 2: a disagreement doesn't name its own cause	3
2.4	Why it is hard, part 3: the world moves too	4
2.5	Why it is hard, part 4: to see yourself, you must move — but moving may be unsafe	4
2.6	The seam: six fields, each holding one piece	5
3	Core Methods	6
3.1	The matched sensor-suite ablation: the one idea that makes the result mean something	6
3.2	Structural sensing and the idea of analytical redundancy	7
3.3	The physics of a bendy arm, and the circularity trap	7
3.4	The estimators, and why there is a ladder of them	8
3.5	From a guess to a safe decision: attribution and calibrated abstention	8
3.6	Closing the loop: does knowing the cause help you recover?	9
4	Evaluation Approach	9
4.1	Two layers, kept apart on purpose	9
4.2	The diagnosis layer	10
4.3	The control layer	10
4.4	Honesty machinery: leakage, pre-registration, and paired uncertainty	10
4.5	The bars, stated once, exactly	11
5	How It All Fits Together	11
5.1	The system as a pipeline	11
5.2	The load-bearing assumptions — where this could break	12

5.3	What this equips you to watch for in Phase 2	13
-----	--	----

1 Introduction

This guide is written for one reader: you, Randy, the director of this project. Its job is not to report what we found — at this point in the project there is nothing found yet — but to hand you the *territory*. By the end you should be able to read the project’s contract (the **Claim Sheet**) without stopping to look things up, and follow the execution phase as an informed participant rather than a spectator watching two agents move pieces around a board.

It helps to hold one distinction in mind from the start. The project asks a big, romantic question — *can a robot sense its own body the way we sense ours?* — but it is deliberately built around a small, unromantic, *testable* version of that question. The romantic version is the horizon; the testable version is the step. Almost everything in this guide is about why the small step is the right one, and what each moving part of it is for.

The small step, in one breath: we build a simulated two-joint robot arm whose links can *bend*, we give it a handful of internal “strain gauges” that report how much its own structure is deforming, and we ask a single narrow question — does that internal structural information let the robot tell *what* just changed about itself (its structure, one of its motors, or one of its own sensors) better than a robot with only conventional sensors can, and does telling them apart actually help it keep doing its job? The rest of this document builds up the concepts you need for that question to feel precise rather than vague: what makes it hard, what methods we use to attack it, how we will measure whether it worked, and how the pieces fit into one system.

I have tied every general idea back to the specific choices in our Claim Sheet as it arrives, because the point of a study guide (as opposed to a textbook) is to make *this* project legible, not to survey a field. Where a concept rests on outside work, there is a link to a credible source and a one-line note on what that source adds beyond what is here.

2 Domain Background: the problem, and why it is hard

2.1 The problem: a body that changes under a controller that assumes it doesn’t

Most robots run on a fixed factory model of their own body — exact link lengths, masses, stiffnesses, and a model of how each motor responds to a command. The control software trusts that model completely, and for good reason: precise control *is* the art of inverting an accurate body model. The trouble is that real bodies do not hold still. Parts wear and loosen; joints heat up and develop friction or backlash; a link takes a knock and goes slightly soft; a motor gradually weakens. And the sensors that are supposed to be watching all of this have their own drift and noise. A robot that keeps trusting its original model can become quietly inaccurate — or unsafe — while remaining perfectly capable, mechanically, of the task in front of it.

Why this matters for Dandelion specifically: general-purpose robots are heading into homes

and small workplaces, exactly the settings where nobody will recalibrate them after every knock, wear event, or hot afternoon. A robot that can notice and absorb its own drift, instead of failing or (worse) carrying on confidently while wrong, is a robot that a normal person can actually afford to keep. That is the mission-level reason the question is worth a project.

The human body is the existence proof that inspired the seed. We stay coordinated as we grow, tire, get injured, or pick up a tool because we have a dense, distributed sense of our own bodies — signals from skin, muscles, tendons, joints, and the balance organs, continuously updating an internal sense of what the body is doing. Robots have almost nothing comparable: a few joint-angle sensors, some motor readings, maybe a camera and one motion sensor. This project takes one concrete slice of that gap — *internal structural sensing* — and asks whether it carries information a conventional robot lacks. We are careful, throughout, to claim only the modest thing (richer *information*) and not the grand thing (that “the body computes”); the field itself draws that line carefully (Müller & Hoffmann, 2017 — a clear philosophical audit of what “morphological computation” does and does not mean, which is why we lean on it to keep our own framing honest).

2.2 Why it is hard, part 1: some changes are invisible in principle

The first difficulty is the deepest, because it is not a matter of building a better estimator — it is a mathematical wall. When a robot moves, there is a mapping from its physical body parameters (masses, stiffnesses, inertias) to the joint-space behavior it can actually observe from its torques and joint motions. A 2024 result proves that this mapping has a *nullspace*: there are combinations of parameter changes — the authors call them “inertial transfers” — that produce *no change at all* in the observable joint-space dynamics (Wensing, Niemeyer & Slotine, 2024 — gives the exact, provable characterization of which parameter changes are identifiable from joint data and which are not; it is the theoretical backbone of our entire motivation). In plain terms: some real changes to the body are *invisible* to joint-only sensing, no matter how clever the software.

This is the single most important idea in the project’s motivation, so it is worth stating its consequence bluntly. If some changes are unobservable from joint data in principle, then *no* amount of estimator capacity fixes them from that data — the only cure is a different *measurement*, one that looks at the body from a physically independent direction. Distributed structural sensing is exactly a bet that strain and curvature along the links supply some of that missing direction. It also sets a fair expectation for the experiment: on changes that lie in the joint-space nullspace, a conventional-suite baseline is *supposed* to fail, and if our experiment never includes such changes, we have rigged the comparison in our own favor. The Claim Sheet’s fault library and its “matched baseline” discipline both trace back to this idea.

2.3 Why it is hard, part 2: a disagreement doesn’t name its own cause

The second difficulty is subtler and is the real technical heart of the project. Suppose a robot compares what it expected to sense with what it actually sensed, and the two disagree. That

disagreement — a “residual,” in the vocabulary of fault diagnosis — tells the robot that *something* is wrong, but not *what*. A weakened motor, a softened link, and a lying joint-sensor can all produce overlapping disagreement patterns. Classical fault-diagnosis theory shows that cleanly separating these causes generally requires either special structural conditions or *added redundancy* — an extra, independent measurement that the ambiguous ones can be checked against (Srivani & Venkata, 2023 — a clean worked example of separating a genuine sensor fault from a real plant disturbance, and of how hard the drift case is; it is the closest baseline to the disambiguation we are attempting).

This reframes the whole project. Our thesis is not that structural sensing is a better sensor in general — it is that structural sensing is precisely the *added, physically-independent redundancy* that lets a robot separate a structural cause from an actuator cause from a sensor cause. That is why the project’s central output is not just “detect a change” but *attribute* it to a source, and why we keep insisting the structural channel must be genuinely independent of the joint signals (more on that when we reach the physics).

2.4 Why it is hard, part 3: the world moves too

The third difficulty comes from the field that has read structural signals the longest: structural health monitoring, the discipline that watches bridges, aircraft, and wind turbines for damage. Its founding lesson is humbling. Damage is a *change* in a structure’s properties, and detecting it always means comparing two states — but the ordinary swings of temperature, load, and operating conditions change those same signals too, often by *more* than real damage does (Farrar & Worden, 2007 — the standard introduction framing damage detection as statistical pattern recognition and naming environmental variation as the central confound; it supplies the vocabulary of “structural change” we borrow). The canonical demonstration is a bridge whose seasonal temperature swing moved its vibration signature by more than a genuine fault would.

For us this is a design constraint, not a footnote. A simulation that idealized away drift and temperature would manufacture a victory that evaporates on contact with reality. So the Claim Sheet deliberately injects realistic sensor drift and temperature cross-sensitivity into *both* the healthy and the faulty data, and treats a benefit that survives those confounds as the only kind that counts. Concretely, the structural sensors we simulate are modeled on fiber-optic strain gauges (fiber Bragg gratings), which have a well-characterized and awkward property: they respond to temperature as well as strain, on the order of ten microstrain of apparent signal per degree Celsius (Silveira et al., 2021 — gives the specific strain and temperature sensitivities we use to make the simulated gauges realistic rather than ideal). Building that nuisance in on purpose is what makes a positive result trustworthy.

2.5 Why it is hard, part 4: to see yourself, you must move — but moving may be unsafe

The fourth difficulty is a genuine tension rather than a wall. To identify what has changed about a body, you need *informative* motion — the system-identification literature calls the ideal condition “persistent excitation.” But the vigorous, wide-ranging motions that best

excite a body’s structural signatures are exactly the motions a possibly-damaged robot ought to avoid. So the project cannot assume it can simply shake the arm hard enough to see everything; it has to ask whether *ordinary task motion* carries enough information, and if not, whether a *short, safe, bounded probing motion* is enough. You will see this show up in the Claim Sheet as two separate excitation conditions, and it is why “excitation budget” is left as an early Phase-2 decision rather than pretended-settled now.

2.6 The seam: six fields, each holding one piece

Put these difficulties together and you can see why the problem has stayed open: it sits in the gap between six research communities that rarely talk to each other, each of which owns one piece and none of which owns the whole. This “seam” is the intellectual center of the project, and it is worth walking once, because the Claim Sheet’s contribution is defined *as* the seam.

- **Robot self-modeling** teaches a robot to learn and revise a model of its own body from experience — the founding demonstration is a robot that re-derives its own shape and recovers after losing a limb (Bongard, Zykov & Lipson, 2006 — the origin of revisable physical self-models; it is the baseline our approach extends). But it senses the body through joints or cameras and *trusts its sensors*, blaming every surprise on the body — which is exactly the assumption a project that must also catch *sensor* faults cannot inherit.
- **Rapid adaptation** compensates for a changed body in a fraction of a second by inferring a compact control signal from recent sensor history (Kumar et al., 2021, “RMA” — the dominant modern adaptation method and our strong competitor; it shows how much a conventional suite plus fast adaptation already achieves). But it deliberately *conflates* all causes into one latent optimized for control, so it cannot say what changed — excellent for robustness, silent on attribution.
- **Online system identification** re-estimates the body’s physics as it runs — but it is the field that discovered the nullspace wall above, so it is bounded in principle from joint data alone.
- **Structural health monitoring** reads exactly the strain and vibration signals we care about — but stops at detection and localization and never closes the loop to control.
- **Soft-robot and tactile proprioception** already put distributed sensing *inside* robots, and one striking recent system predicts its own shape and reads the mismatch as external contact (Wang et al., 2024 — the closest existing mechanism to ours, a self-model residual doing double duty; it shows the residual idea works, pointed at contact rather than health). But this community uses internal sensing to estimate *shape and touch*, not *health* or fault source.
- **Sensor-fault diagnosis** worries about whether the sensors themselves are lying — but, as we saw, a residual names disagreement, not cause.

Lay them side by side and the unoccupied middle is *source attribution under a revisable self-model*: when the command-to-sensor-to-motion relationship shifts, was it the structure,

an actuator, or the sensor — and does distributed structural sensing supply the redundancy that answers this *and* yields a control advantage? That question is the project. Notice that the contribution is not inventing any one of the six pieces; it is a narrow, controlled test of whether *connecting* them — specifically, adding the structural piece — buys something measurable.

3 Core Methods

3.1 The matched sensor-suite ablation: the one idea that makes the result mean something

Start with the experimental logic, because every other method choice serves it. The danger in a project like this is subtle: if we build a robot with structural sensing, give it a good learning algorithm, and show it works well, we will have proven almost nothing — because we will not know whether the success came from the *sensors* or from the *algorithm*. The fix is an *ablation*: hold the algorithm, the controller, the training data, the faults, and the motions all *identical*, and vary *only* the sensors. Then any measured difference in performance is attributable to the information the sensors carry, not to a difference in the software reading them. This is the backbone of the whole design, and it is why the Claim Sheet calls the sensor suite “the controlled variable.”

Concretely, we compare four *nested* sensor suites, each a superset of the previous one:

- **C0** — joint-angle sensors plus the motor commands. The lean conventional floor.
- **C1** — C0 plus a noisy motor-current reading (converted into a rough effort estimate using the motor’s nominal spec) plus one six-axis motion sensor (an IMU) on the far link. This is meant to be the *richest onboard suite a cheap real robot plausibly already carries* — the honest, strong conventional baseline.
- **S** — C1 plus the four structural strain/curvature gauges. This is the suite under test.
- **O** — an “oracle” that can read the simulator’s hidden true state. It is an unbeatable ceiling, never something a real robot could have; it exists to tell us how much *any* method leaves on the table.

Two design decisions here are load-bearing and worth flagging, because they came out of the Claim Sheet’s review and both protect the honesty of the comparison. First, the real question is whether S beats the *strongest fair* conventional suite, C1 — not whether it beats a strawman. Beating a deliberately-thin baseline (say, C0 alone) would be a hollow win, so C1 is built up to be genuinely strong. Second, C1 gets a *noisy current-derived estimate* of motor effort, not the true delivered torque, and the actuator-weakening fault is applied *downstream* of that reading. This matters because if C1 could read the true delivered torque, it would be handed the answer to “did the motor weaken?” for free, and the actuator-attribution question would be trivial. Keeping the true torque out of C1 is what keeps the question real.

3.2 Structural sensing and the idea of analytical redundancy

Now the sensing itself. “Strain” is how much a material is locally stretched or compressed; “curvature” is how much a link is locally bent. On a body whose links actually flex, these are real physical quantities that carry information about the loads flowing through the structure — the same quantities aerospace engineers already instrument for. We simulate four fixed measurement stations, two per link, each reporting local bending strain/curvature, standing in for what an embedded fiber-optic array could measure on a future real robot. Their positions are fixed *before* any learning and held fixed for the comparison, so we cannot cheat by tuning sensor placement on the test set.

The key concept these sensors are meant to provide is *analytical redundancy*: an extra, physically-independent view of the body that ambiguous signals can be checked against. Recall the disambiguation problem — a residual names disagreement, not cause. If the strain history says the link is bending in a way that is inconsistent with what the (possibly-lying) joint sensor reports, that inconsistency is evidence that points at the *sensor* rather than the structure. If instead the strain pattern itself shifts in a repeatable way, that points at a *structural or actuator* change. The whole bet is that this independent view turns an under-determined “something changed” into a better-determined “*this* changed.” This is why the Claim Sheet’s success condition is about *attribution*, not mere detection, and why realism in the strain channel is non-negotiable: analytical redundancy is only real if the redundant channel is genuinely independent and genuinely noisy in the ways a real one would be (Barrias et al., 2016 — a survey of distributed fiber-optic strain sensing that grounds the resolutions and the “many channels cross-checking each other” idea we rely on).

3.3 The physics of a bendy arm, and the circularity trap

To have real strain, we need a body that really bends, so the plant is a two-link arm whose links are modeled as flexible beams rather than rigid bars. How we simulate that flex is where the most technically delicate decision lives, and it is guarded by what the Claim Sheet calls the *feasibility spike* — a gate we must pass before committing to any engine.

The trap the spike guards against is *circularity*. If the “strain” we feed the estimator is secretly just an algebraic re-encoding of the joint angles the conventional suite already has, then S has no independent information and any apparent win is an artifact. The defense is to derive the strain from *independent deformation coordinates* — degrees of freedom the simulator integrates from applied forces, physically distinct from the joint coordinates — and then cross-check them against a completely independent physics calculation. Our primary engine candidate is MuJoCo, a fast, free, widely-used physics engine (MuJoCo — the engine and its documentation; it supplies the rigid-body dynamics and the deformable elements we test). A correction from the Claim Sheet review is worth internalizing because it is exactly the kind of thing that separates a careful build from a naive one: MuJoCo’s generic “flex” element is primarily a *stretchable line*, not automatically a bending beam or a strain sensor, so the spike must test the engine’s actual bending-capable mechanics (its cable/rod path, and a slender solid model only if needed) rather than assume any “flex” will do.

If the native mechanics cannot produce clean, independent, above-the-noise fault signatures

at realistic (metal-like) stiffness, we fall back to a Cosserat-rod model, a well-established way to simulate the bending and twisting of slender rods (PyElastica — an open Cosserat-rod simulator that both serves as a fallback plant and as the independent calculation we validate the strain against). Heavy finite-element simulation stays offline, used only to generate ground truth and validate the cheaper models, never inside the live control loop — it is too slow for that and would blow the compute budget. This is the “fidelity ladder” in the Claim Sheet: native rod, then reduced-order Cosserat, then offline finite-element ground truth.

3.4 The estimators, and why there is a ladder of them

The “brain” that reads a sensor suite and calls the fault could be built many ways, and the Claim Sheet deliberately runs several, because each answers a different worry.

The primary vehicle is a *compact temporal neural network* — a small model that reads a short window of recent sensor history and outputs a guess at the current fault and its severity. It is temporal because a single instant rarely reveals a fault; the pattern over time is what carries the signal. This same network is run identically on C0, C1, and S — it *is* the ablation.

But a neural network alone invites a fair objection: maybe a win just means “a bigger black box won.” So the Claim Sheet pairs it with a *simple, interpretable baseline* — a residual or linear system-identification estimator — as a transparent floor. If the interpretable method already extracts the signal, we learn the effect is real and not a capacity artifact; if only the bigger model finds it, we learn something about what representation the information lives in.

The most important comparator is the *strong* one, an RMA-style latent adapter. This is the deflationary competitor: it receives the same conventional sensor history and adapts fast, but optimizes an internal control code rather than a labeled fault call. It embodies the hypothesis we most need to rule out — that ordinary sensors plus fast adaptation *already* recover whatever structural sensing was supposed to add. If RMA-on-C1 matches S, the honest conclusion is that structural sensing was not necessary, and the Claim Sheet treats that as a real (negative) result rather than something to explain away. Building your strongest opponent on purpose is what makes a positive result credible; this is worth watching for in Phase 2 as a sign the team is keeping itself honest.

Finally there is the oracle controller (simply told the true fault) as a ceiling, and a famous “recover like an animal” method (Cully et al., 2015 — recovers function after damage *without* diagnosing the cause; we include it as a reference point precisely because it defines the “don’t bother attributing, just adapt” philosophy our question is testing against) included only as a reference, since it plays by different computational rules and would not be a fair matched baseline.

3.5 From a guess to a safe decision: attribution and calibrated abstention

A fault call is only useful if the robot knows *when to trust it*. Acting confidently on a wrong diagnosis — reconfiguring the controller for a motor fault that is actually a sensor glitch — can be worse than doing nothing. So attribution in this project is explicitly *staged and*

calibrated: detect that something changed; classify it as structure, actuator, or sensor — *or abstain* and say “I’m not sure”; localize and estimate severity where possible; and only then use the estimate for recovery. The ability to abstain is treated as first-class, not as a failure to answer, because an honest “unknown” is often the safe action.

This makes *calibration* a real requirement rather than a nicety. A well-calibrated system’s confidence means what it says: among the cases where it claims 80% certainty, it should be right about 80% of the time. And an “allowed to abstain” system has to be measured carefully so that abstaining cannot secretly inflate its score — a system that only answers the easy cases and abstains on the hard ones looks artificially accurate. Recent work lays out exactly how to evaluate such systems honestly, separating raw probability quality from the accept/reject trade-off from the detection of genuinely novel inputs (Traub et al., 2024 — the source of the specific abstention-evaluation discipline the Claim Sheet adopts, so rejection cannot manufacture a diagnosis win). When you reach the Claim Sheet’s evaluation slot and see calibration, risk-coverage, and unknown-detection reported as three *separate* things, this is why.

3.6 Closing the loop: does knowing the cause help you recover?

The last method is the one that earns the word “advantage.” It is not enough for structural sensing to *name* the cause better; the project only claims a win if naming the cause *helps the robot recover the task*. So the controller is reconfigured based on the estimated fault — reweighting or rescheduling its behavior for the specific change it believes has occurred — and this is compared against a control-only adapter that gets the same history but no explicit “name the cause” objective. That contrast isolates the thing we actually care about: does *knowing what changed* buy better recovery than simply *adapting to* the fact that something did? It is entirely possible for the answer to be “no, attribution doesn’t help control” even if attribution improves — which is a specific, pre-declared outcome we will meet in the evaluation section.

4 Evaluation Approach

4.1 Two layers, kept apart on purpose

The single most important thing to understand about how we measure success is that there are *two separate layers* and we never blend them into one score. The *diagnosis* layer asks: did the robot name the cause correctly? The *control* layer asks: did it recover the task better? These are kept apart because they can genuinely diverge — structural sensing might improve diagnosis while making no difference to control, and if we collapsed both into a single number that real and interesting outcome would be hidden. The Claim Sheet’s evaluation slot is organized around this split, and so is its list of possible outcomes.

4.2 The diagnosis layer

The headline diagnosis number is *four-way macro-F1* across the classes {healthy, structure, actuator, sensor}. F1 is a standard accuracy score that combines two kinds of mistake — false alarms and misses — into a single value between 0 (worst) and 1 (best); “macro” means we average it evenly across the four classes so that a rare cause counts as much as a common one, and the robot cannot look good just by nailing the easy majority class ([scikit-learn’s f1_score documentation](#) — a precise, runnable definition of F1 and macro-averaging). Around that headline sit per-class confusion tables (with special attention to the genuinely hard confusions, structure-versus-actuator and structure-versus-sensor), detection delay, localization error, and — because a fault call needs honest confidence — calibration, measured with standard scores and reliability diagrams ([scikit-learn’s probability-calibration guide](#) — explains reliability diagrams and what “well-calibrated” means operationally). The abstention behavior is measured on its own terms, and the never-before-seen compound fault is used as a test of whether the system can flag a genuinely unfamiliar situation as unknown rather than forcing it into one of the four boxes.

4.3 The control layer

The headline control number is the *five-second post-change integral of absolute tracking error*. In words: over the five seconds right after a change is applied, add up how far the arm is from where it should be at each instant. Precisely, and this is the one place a formula genuinely clarifies rather than decorates:

$$J_{5s} = \int_{t_c}^{t_c+5} \|e(t)\| dt,$$

where t_c is the instant the change happens, $e(t)$ is the tracking error at time t (the gap between where the arm is commanded to be and where it actually is), $\|\cdot\|$ turns that gap into a single non-negative size, and the integral simply accumulates that size over the five-second recovery window. Lower is better — it means the arm spent those five seconds closer to correct. The success bar (below) asks S to cut this quantity by at least 10% relative to C1. Alongside it we report tracking error before, during, and after adaptation, recovery time, how much of healthy performance is regained, and any unsafe or constraint-violating excursions, because a “recovery” bought by dangerous motion is not a recovery.

4.4 Honesty machinery: leakage, pre-registration, and paired uncertainty

Three disciplines keep the numbers from lying to us, and they are worth understanding because they are where data-driven results in this area most often go wrong.

Leakage-free splitting. We divide data into development, pilot, validation, and final-test sets by *whole runs and whole fault settings* — never by chopping a single run into training and testing pieces. If pieces of the same run appeared on both sides, the model could pass by memorizing that run’s fingerprint rather than learning the fault, and report a score that collapses the moment it meets genuinely new data. This exact failure has embarrassed a

widely-used benchmark in a neighboring field ([Hendriks et al., 2022](#) — shows that a standard fault-diagnosis benchmark’s usual split reuses the same physical hardware across train and test, inflating accuracy dramatically; it is our cautionary tale for why the split discipline is strict).

Pre-registration. Before we generate the final confirmatory data, we *freeze* everything that could be quietly tuned to flatter the result — sensor placement, model settings, decision thresholds, the analysis window, and the full list of scenarios and random seeds — into a versioned configuration. And critically, the success *thresholds themselves* are fixed in advance, chosen as the smallest effects we would consider practically meaningful, before any pilot data is seen. The pilot’s only job is to tell us how many runs we need to measure such an effect reliably; it is explicitly *not* allowed to choose what counts as success. This is what stops the all-too-common move of looking at the results and then deciding what bar they happen to clear.

Paired, hierarchical uncertainty. Because S and C1 are tested on the *same* scenarios, we compare them *paired* — scenario by scenario — which cancels a lot of nuisance variation and makes the comparison sharper. We put an honest uncertainty range on the result with a *hierarchical bootstrap*, a standard resampling method that respects the nested structure of the data (many scenarios within each random seed) rather than pretending every data point is independent ([SciPy’s bootstrap documentation](#) — a runnable reference for bootstrap confidence intervals, the tool we use to say “the interval excludes zero”). A result “counts” only when that paired interval excludes zero.

4.5 The bars, stated once, exactly

Putting the layers and the machinery together, here is the full success condition, fixed in advance: S improves four-way macro-F1 over C1 by **at least 0.05 absolute**, with the paired 95% interval excluding zero and *no* single cause doing worse than a small -0.02 margin (a “non-inferiority” guard, borrowed from clinical trials, meaning “allowed to be no worse than this much” — it stops an average gain from hiding a regression on one cause); *and* S reduces the five-second post-change tracking-error integral by **at least 10%**, paired interval excluding zero, with no safety regression; *and* both hold under realistic drift, temperature, and held-out-severity confounds. All of it is required. Anything less — diagnosis improves but control does not, or the benefit appears only for one change type, or only under idealized sensors, or only on data resembling the training data — is a genuine but *partial* outcome, and the Claim Sheet names each of those partial shapes in advance so a partial win can never be reported as a full one.

5 How It All Fits Together

5.1 The system as a pipeline

Step back and the project is one pipeline with four stages, and it helps to see what each stage receives, transforms, and passes on.

The *plant* (the simulated bendy arm) receives motor commands and produces the true physical state — joint motions, and the independent deformation coordinates from which real strain is computed. The *signals* stage receives that true state and produces what the robot actually gets to see: it injects a fault (softening a link, weakening a motor, or corrupting a sensor) and then applies realistic sensor imperfections (noise, drift, temperature cross-sensitivity, dropout), emitting the observed C0/C1/S/O suites. The *estimator* receives a short causal window of one suite’s observations and produces a calibrated fault call — class, confidence, and, where possible, location and severity — or an honest abstention. The *controller* receives that estimate and reconfigures the robot’s behavior to recover the task. The evaluation then reads the whole chain on two axes: did the estimator name the cause (diagnosis layer), and did the controller recover (control layer)?

Two features of this pipeline are deliberate and load-bearing. First, the boundary between the plant’s *true* state and the suites’ *observed* state is kept strict: only the oracle and the ground-truth labels may read the privileged true state, precisely so that no privileged information leaks into a deployable suite and manufactures a false advantage. Second, the interface between these stages — what the plant emits, what the signals stage adds, what the estimator consumes — is being agreed and *versioned as a shared schema before either agent writes implementation code*, so that the two work streams (the physics/plant/control lane and the sensing/diagnosis/evaluation lane) meet cleanly rather than after the fact. That coordination detail is not bureaucracy; it is what lets a two-agent team build one coherent system instead of two halves that do not fit.

5.2 The load-bearing assumptions — where this could break

Good systems thinking means naming the assumptions the whole edifice rests on, because those are where it will break if it breaks. Here are the ones to watch, and each maps to something concrete in the Claim Sheet:

- **The structural channel is genuinely independent.** If simulated “strain” is secretly an algebraic echo of the joint angles, S has no new information and the experiment is void. *Guarded by* the feasibility spike’s independent-deformation-coordinate requirement and the independent-physics cross-check. This is the assumption most likely to sink the project quietly, which is why it is gated first.
- **The confounds are modeled, not idealized.** A benefit that only exists with perfect sensors is not a benefit. *Guarded by* building drift and temperature cross-sensitivity into both healthy and faulty data and requiring the effect to survive them.
- **The conventional baseline is genuinely strong.** Beating a strawman proves nothing. *Guarded by* building C1 up to the richest fair onboard suite and keeping the true delivered torque out of it so the actuator question stays hard.
- **The effect must generalize.** An in-distribution-only win is fragile. *Guarded by* held-out severities, motions, payloads, and an unseen compound fault.
- **Informative motion is available without being unsafe.** If only unsafe excitation reveals the faults, the approach is impractical. *Guarded by* testing ordinary-motion and

short-safe-probe conditions separately.

- **The bar was set before the result.** A movable bar can always be cleared. *Guarded by* pre-registration and the pilot-sizes-not-chooses rule.

5.3 What this equips you to watch for in Phase 2

With this territory in hand, you can follow the execution phase as a participant. The first thing to watch is the feasibility spike: does the simulated arm produce clean, independent, above-the-noise fault signatures at realistic stiffness? If it does not, the honest move is the fidelity ladder or an amended plant — not a quiet weakening of the claim, and you now know why. After that, the questions become: does S actually beat C1 on diagnosis, and does that survive the confounds and the held-out tests? And if diagnosis improves, does it translate into recovery, or do we land in the “diagnostic-only” outcome? You will be able to read each result against a bar that was fixed in advance, and to tell a full success from a bounded one from a clean, useful negative — which is exactly the position a director needs to be in to judge whether the work is real.

That judgment is the whole reason this guide exists. The romantic horizon — robots that sense their own bodies — is worth keeping in view, but this project earns its place by taking one honest, checkable step toward it and refusing to claim more than the step supports. When the results arrive, a second pass of this guide will give you the extra vocabulary the execution surfaced; for now, you are equipped to follow the contract and the work it commits to.